## Task 1: Watch the Terraform-05 video.

## Task 2: Execute the script shown in the video.

## Task: Create an S3 Bucket Using Terraform

# Objective

To create an Amazon S3 bucket using Terraform in VS Code and understand how to provision AWS resources using Infrastructure as Code (IaC).

# Terraform Code — `main.tf`

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "s3_bucket" {
  bucket = "s3backend-asif-terraform-2026"

  tags = {
    Name = "Terraform-Remote-State"
  }
}
```

# Steps Used

1. Created the project directory `C:\repos\Terraform-01` in VS Code.
2. Created the `main.tf` file.
3. Configured the AWS provider with region `us-east-1`.
4. Defined the S3 bucket resource with a globally unique bucket name.
5. Added a tag to identify the bucket.
6. Initialized the Terraform working directory:

```
terraform init
```

7. Validated the configuration:

```
terraform validate
```

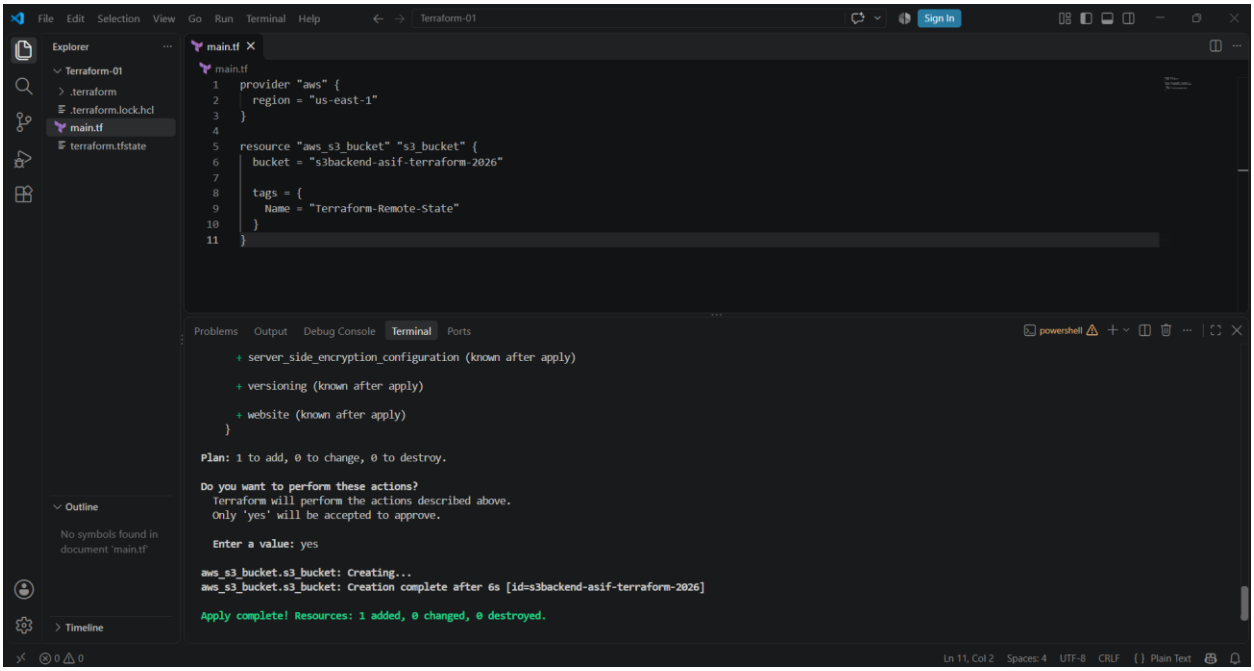
8. Previewed the infrastructure changes:

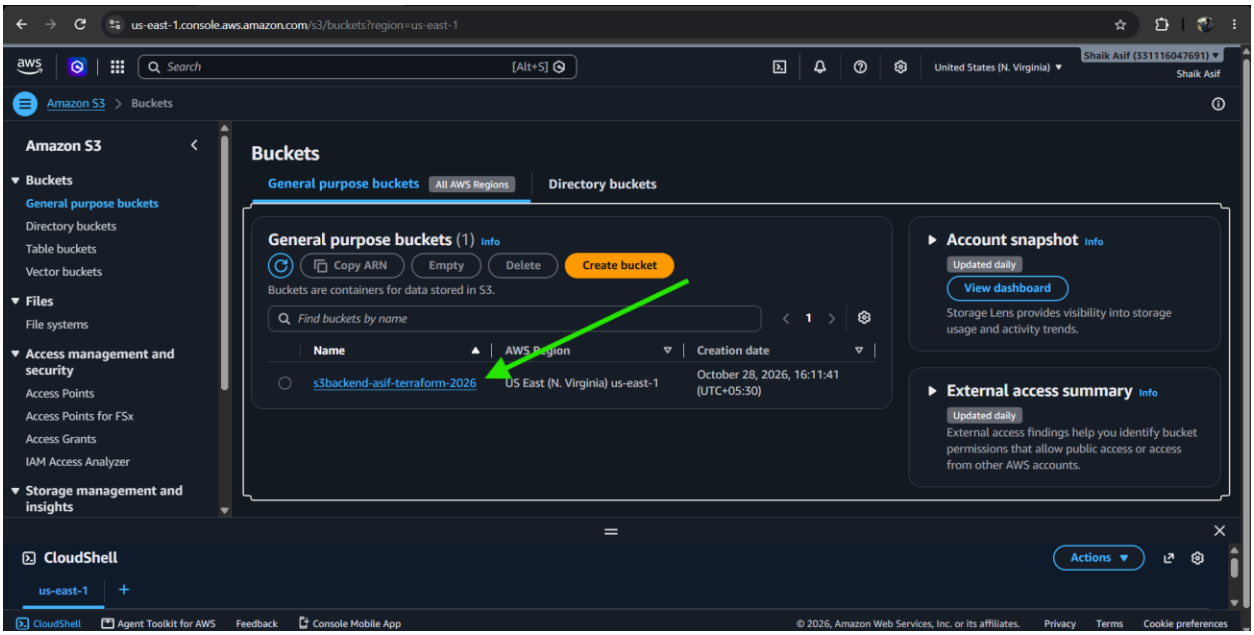
```
terraform plan
```

9. Created the S3 bucket:

```
terraform apply
```

10. Entered `yes` to approve the changes and verified that the bucket was created successfully.






# Observation

- Terraform initialized successfully and installed the AWS provider.
- Configuration validation returned `Success! The configuration is valid.`
- The plan showed one resource to be created.
- Terraform created the S3 bucket successfully in `us-east-1`.
- Final result: `1 added, 0 changed, 0 destroyed`.

# Conclusion

Successfully created an Amazon S3 bucket using Terraform. This task demonstrated the use of the AWS provider, S3 resource configuration, and the Terraform initialization, validation, planning, and application workflow.

# Task: Create a DynamoDB Table Using Terraform

## Objective

To create an Amazon DynamoDB table using Terraform and understand how it can be used in the traditional Terraform remote-state locking approach.

## Terraform Code — `main.tf`

The DynamoDB resource was added below the existing S3 resource in the same Terraform configuration.

```
resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {
  name         = "terraform-state-lock-dynamo"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"

  attribute {
    name = "LockID"
    type = "S"
  }
}
```

## Steps Used

1. Continued working in the existing project directory, `C:\repos\Terraform-01`.
2. Added the DynamoDB resource block to `main.tf`.
3. Configured the table name as `terraform-state-lock-dynamo`.
4. Set `PAY_PER_REQUEST` billing mode.
5. Configured `LockID` as the partition key with type `S` (String).
6. Initialized the Terraform working directory:

```
terraform init
```

7. Formatted the configuration:

```
terraform fmt
```

8. Validated the configuration:

```
terraform validate
```

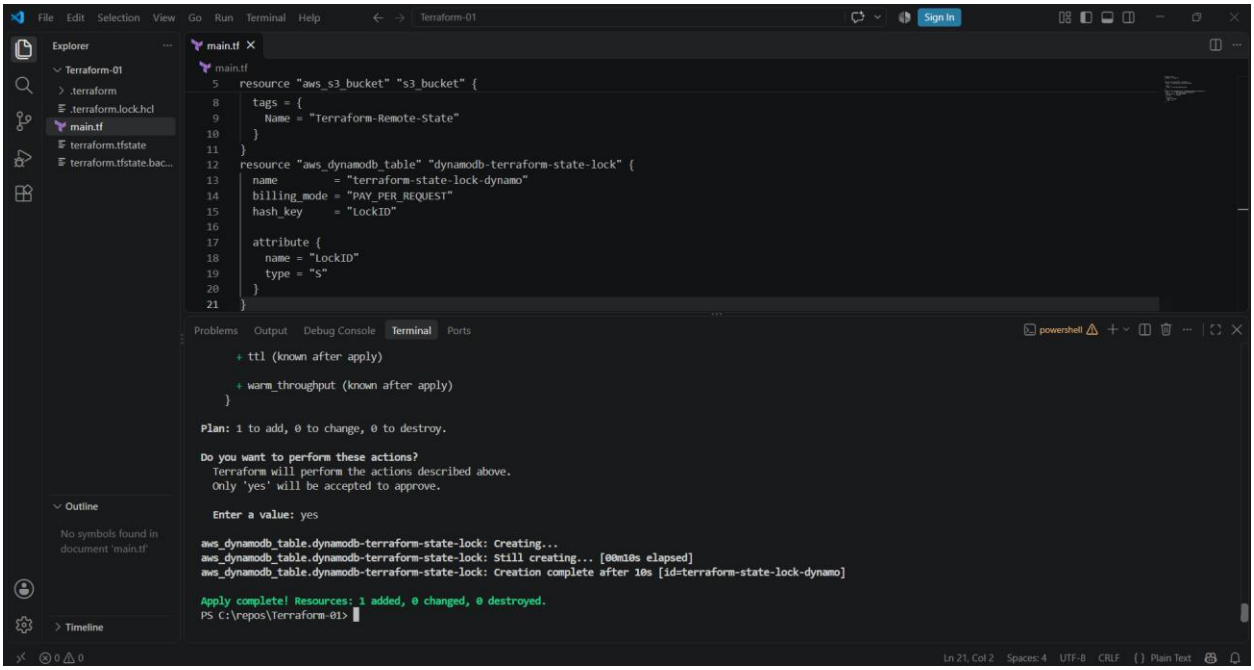
9. Reviewed the planned changes:

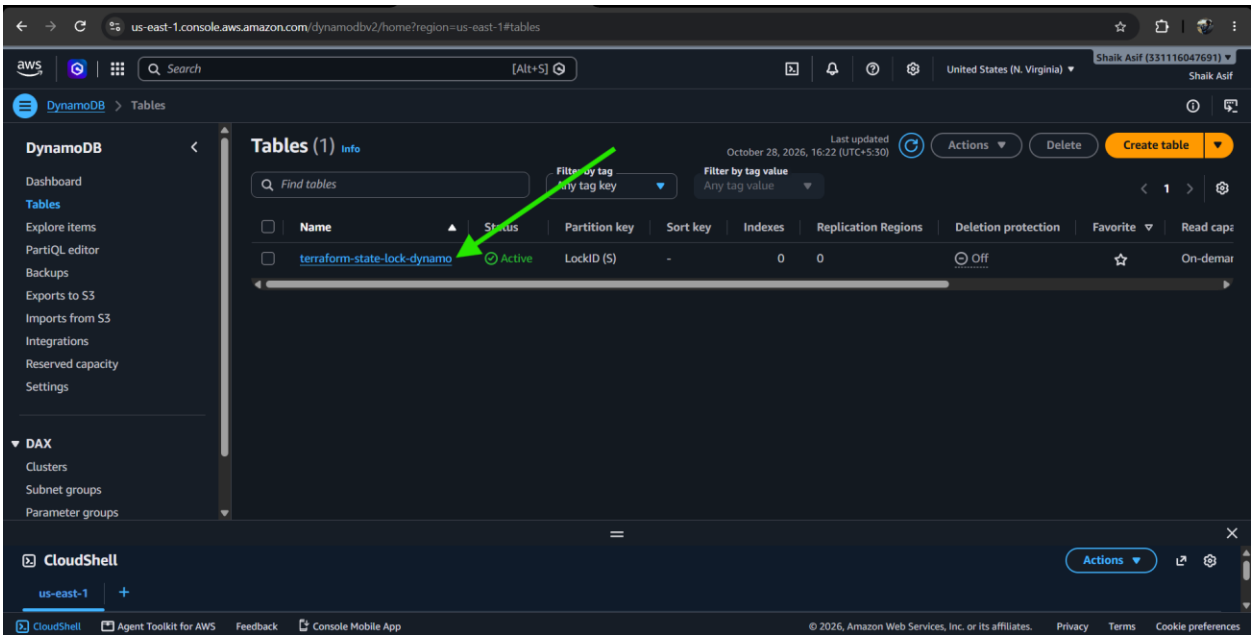
```
terraform plan
```

10. Applied the configuration:

```
terraform apply
```

11. Entered `yes` to approve the changes and confirmed successful table creation.






# Observation

- Terraform initialized successfully and reused the installed AWS provider.
- Formatting and validation completed successfully.
- The plan showed only the DynamoDB table to be created.
- The existing S3 bucket remained unchanged.
- Terraform created the DynamoDB table successfully.
- Final result: `1 added, 0 changed, 0 destroyed`.

# Conclusion

Successfully created an Amazon DynamoDB table using Terraform. The table uses a string partition key named `LockID` and on-demand billing. This task demonstrated how Terraform provisions a DynamoDB table that can be used to study the traditional state-locking approach.

# Task: Configure Terraform Remote Backend Using Amazon S3

## 1. Objective

To configure Amazon S3 as a remote backend for Terraform state storage, migrate the existing local state file to S3, and verify that Terraform can manage infrastructure using the remote state.

## 2. Code Used

**File name:** `main.tf`

```
terraform {
  backend "s3" {
    bucket       = "s3backend-asif-terraform-2026"
    key          = "remote-state/terraform.tfstate"
    region       = "us-east-1"

    encrypt      = true
    use_lockfile = true
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "s3_bucket" {
  bucket = "s3backend-asif-terraform-2026"

  tags = {
    Name = "Terraform-Remote-State"
  }
}

resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {
  name         = "terraform-state-lock-dynamo"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"
```

```
  attribute {
    name = "LockID"
    type = "S"
  }
}
```

### Backend configuration explanation

- `bucket`: Specifies the S3 bucket where Terraform state is stored.
- `key`: Specifies the location of the state file inside the bucket.
- `region`: Specifies the AWS region.
- `encrypt = true`: Enables server-side encryption for the state object.
- `use_lockfile = true`: Enables S3-native state locking using a lockfile object.

**Note:** The DynamoDB table is defined as a resource for practice, but this configuration uses S3-native locking rather than DynamoDB locking.

## 3. Steps Followed

1. Opened the existing project directory `C:\repos\Terraform-01` in PowerShell and VS Code.
2. Created the S3 bucket `s3backend-asif-terraform-2026` using Terraform.
3. Created the DynamoDB table `terraform-state-lock-dynamo` with on-demand billing.
4. Backed up the existing Terraform state using:

```
terraform state pull > terraform-state-backup.json
```

5. Added the `terraform` backend block to the existing `main.tf` file.
6. Initialized Terraform:

```
terraform init
```

7. When Terraform prompted to migrate the existing local state to S3, entered `yes`.
8. Formatted the configuration:

```
terraform fmt
```

9. Validated the configuration:

```
terraform validate
```

10. Checked the infrastructure plan:

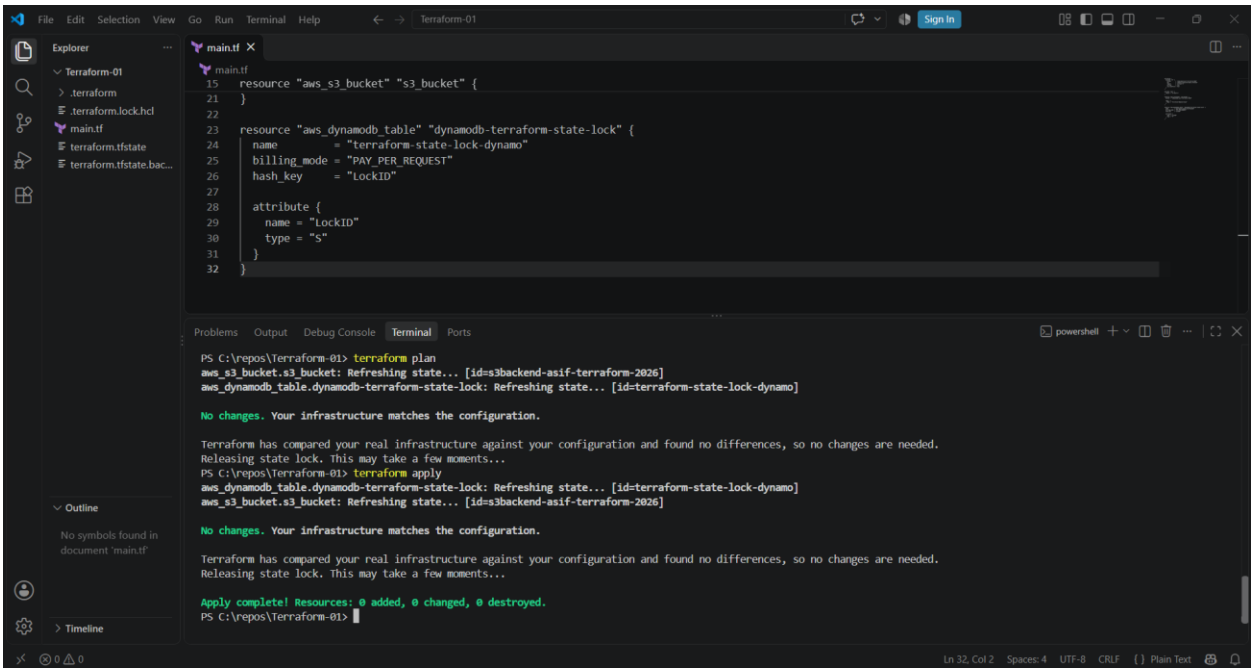
```
terraform plan
```

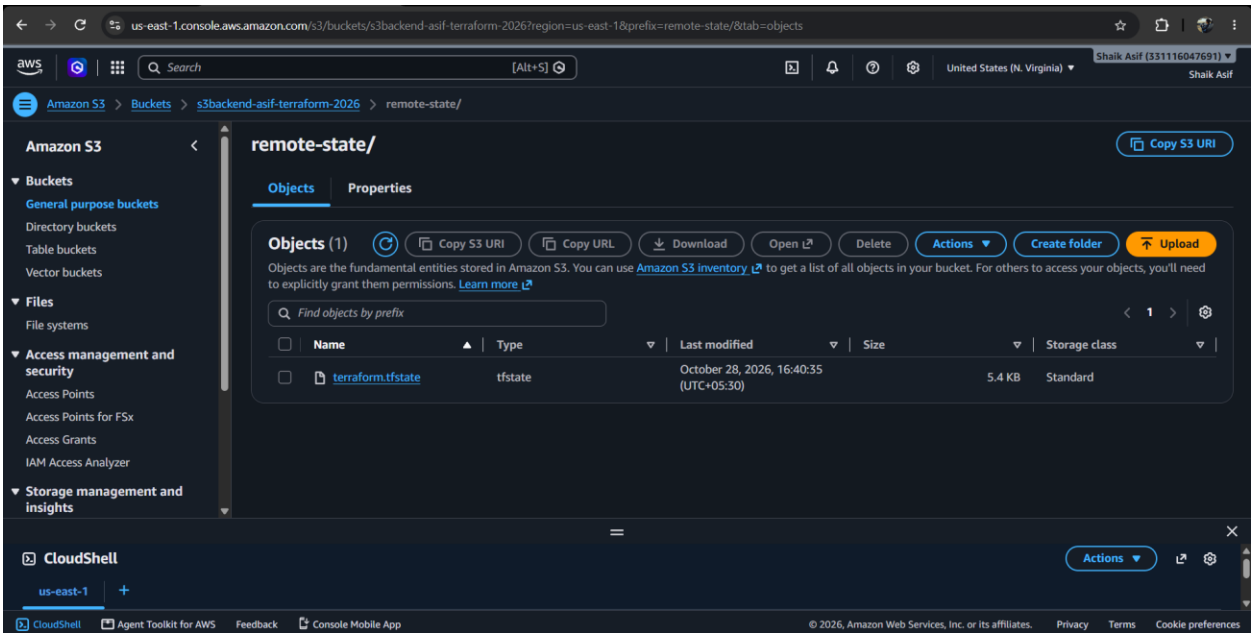
11. Applied the configuration:

```
terraform apply
```

12. Verified the remote state object using AWS CLI:

```
aws s3api head-object --bucket s3backend-asif-terraform-2026 --key
remote-state/terraform.tfstate --region us-east-1
```






# 4. Observation

- Terraform initialized successfully with the S3 backend.
- The existing local state was migrated to the remote backend.
- `terraform validate` returned `Success! The configuration is valid.`
- `terraform plan` reported that no infrastructure changes were required.
- `terraform apply` completed with **0 added, 0 changed, and 0 destroyed**.
- The AWS CLI successfully returned metadata for the remote state object.
- The backend uses S3-native locking through `use_lockfile = true`.

# 5. Conclusion

Successfully configured Amazon S3 as the Terraform remote backend and migrated the existing state file from local storage to S3. Verified that Terraform can read the existing infrastructure state and that no unintended infrastructure changes were required.

Remote state storage makes the state file accessible to authorized team members and supports collaboration. S3-native locking helps prevent conflicting Terraform operations when multiple users or processes work with the same state.

# Task: Terraform local-exec Provisioner

## 1. Objective

To understand and implement the Terraform `local-exec` provisioner to execute a command on the local machine during resource creation, without creating any additional AWS infrastructure.

## 2. Code Used

**File name:** `local-exec.tf`

```
resource "terraform_data" "local_exec_demo" {
  provisioner "local-exec" {
    command = "echo Local-exec provisioner executed successfully!"
  }
}
```

### Code Explanation

- **`terraform_data`**: A built-in Terraform resource used to manage data and provisioner demonstrations without creating an AWS resource.
- **`local_exec_demo`**: The local name assigned to the resource.
- **`provisioner "local-exec"`**: Executes a command on the machine running Terraform.
- **`command`**: Specifies the command to execute. Here, it prints a success message in the terminal.

## 3. Steps Followed

**Step 1: Opened the project**

Opened the existing Terraform project directory in PowerShell:

```
cd C:\repos\Terraform-01
```

**Step 2: Created the configuration file**

Created a new file named `local-exec.tf` and added the `terraform_data` resource with the `local-exec` provisioner.

**Step 3: Initialized Terraform**

```
terraform init
```

Terraform initialized successfully and loaded the required built-in and AWS providers.

**Step 4: Validated the configuration**

```
terraform validate
```

Output:

```
Success! The configuration is valid.
```

**Step 5: Reviewed the execution plan**

```
terraform plan
```

Terraform displayed:

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

**Step 6: Applied the configuration**

```
terraform apply
```

Confirmed the operation by entering `yes`.

The provisioner executed the command and printed:

```
Local-exec provisioner executed successfully!
```

Terraform completed successfully:

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

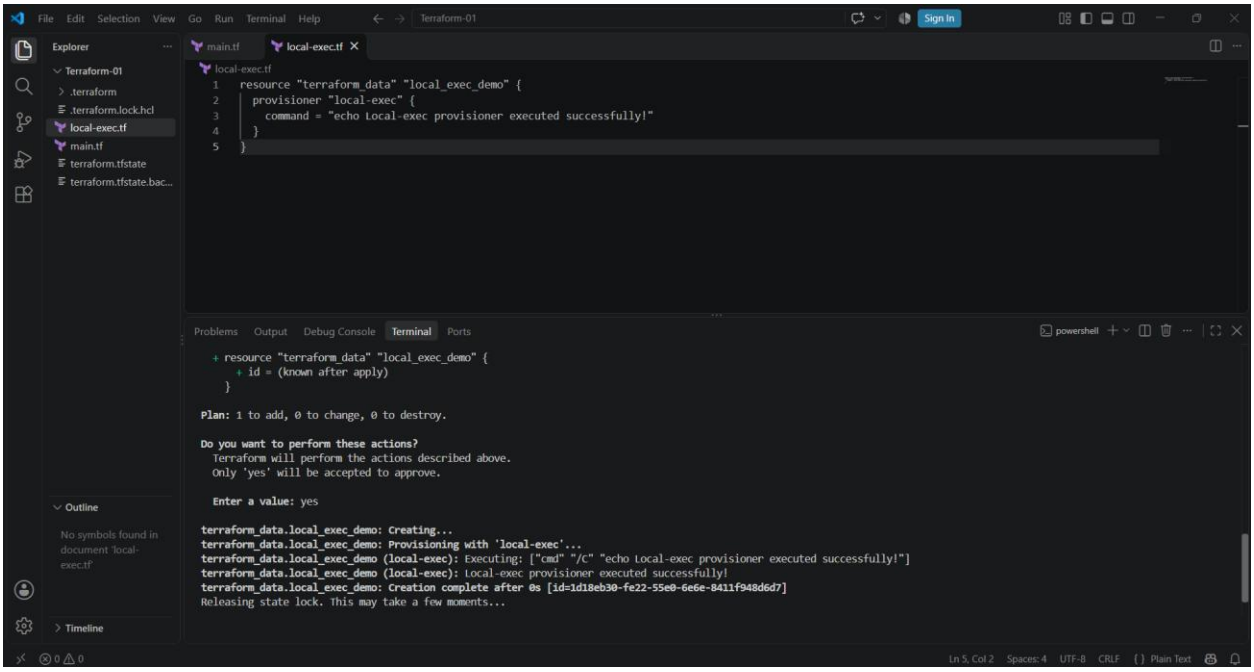
**Step 7: Verified the Terraform state**

```
terraform state list
```

Output:

```
aws_dynamodb_table.dynamodb-terraform-state-lock
aws_s3_bucket.s3_bucket
terraform_data.local_exec_demo
```

This confirmed that Terraform tracks the new `terraform_data` resource alongside the existing S3 bucket and DynamoDB table in the remote state.




# 4. Observation

- Terraform initialized and validated the configuration successfully.
- The execution plan identified one resource for creation.
- The `local-exec` provisioner executed the command on the local Windows machine using `cmd /C`.
- The expected success message appeared in the terminal.
- Terraform recorded `terraform_data.local_exec_demo` in the state.
- No additional AWS infrastructure was created by this task.

# 5. Conclusion

Successfully implemented and executed the Terraform `local-exec` provisioner. Learned how Terraform can execute local operating-system commands during resource creation.

The `local-exec` provisioner is useful for tasks such as printing messages, running local scripts, or invoking commands as part of a Terraform workflow. However, provisioners should generally be used only when a suitable native Terraform resource or other configuration mechanism is not available.

# Task: Terraform Resource Replacement Using `-replace`

## 1. Objective

To understand how to replace an existing Terraform resource using the modern `terraform plan -replace` and `terraform apply -replace` commands, without modifying existing AWS infrastructure.

## 2. Code Used

**File name:** `local-exec.tf`

```
resource "terraform_data" "local_exec_demo" {
  provisioner "local-exec" {
    command = "echo Local-exec provisioner executed successfully!"
  }
}
```

### Code Explanation

- `terraform_data`: A built-in Terraform resource that can be used without creating an AWS resource.
- `local_exec_demo`: The resource's local name.
- `local-exec`: Executes a command on the machine running Terraform.
- `command`: Prints a success message in the terminal.

## 3. Steps Followed

### Step 1: Opened the project directory

```
cd C:\repos\Terraform-01
```

Used the existing Terraform project and its S3 remote backend.

### Step 2: Checked existing resources

```
terraform state list
```

Confirmed that the following resources were tracked:

```
aws_dynamodb_table.dynamodb-terraform-state-lock
aws_s3_bucket.s3_bucket
terraform_data.local_exec_demo
```

### Step 3: Planned the resource replacement

```
terraform plan -replace="terraform_data.local_exec_demo"
```

Terraform displayed the replacement plan:

```
-/+ destroy and then create replacement

Plan: 1 to add, 0 to change, 1 to destroy.
```

This command only displayed the proposed changes; it did not replace the resource.

### Step 4: Applied the resource replacement

```
terraform apply -replace="terraform_data.local_exec_demo"
```

Entered `yes` when prompted to confirm the operation.

Terraform destroyed the existing resource and created a replacement.

The `local-exec` command ran again:

```
Local-exec provisioner executed successfully!
```

The final result was:

```
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
```

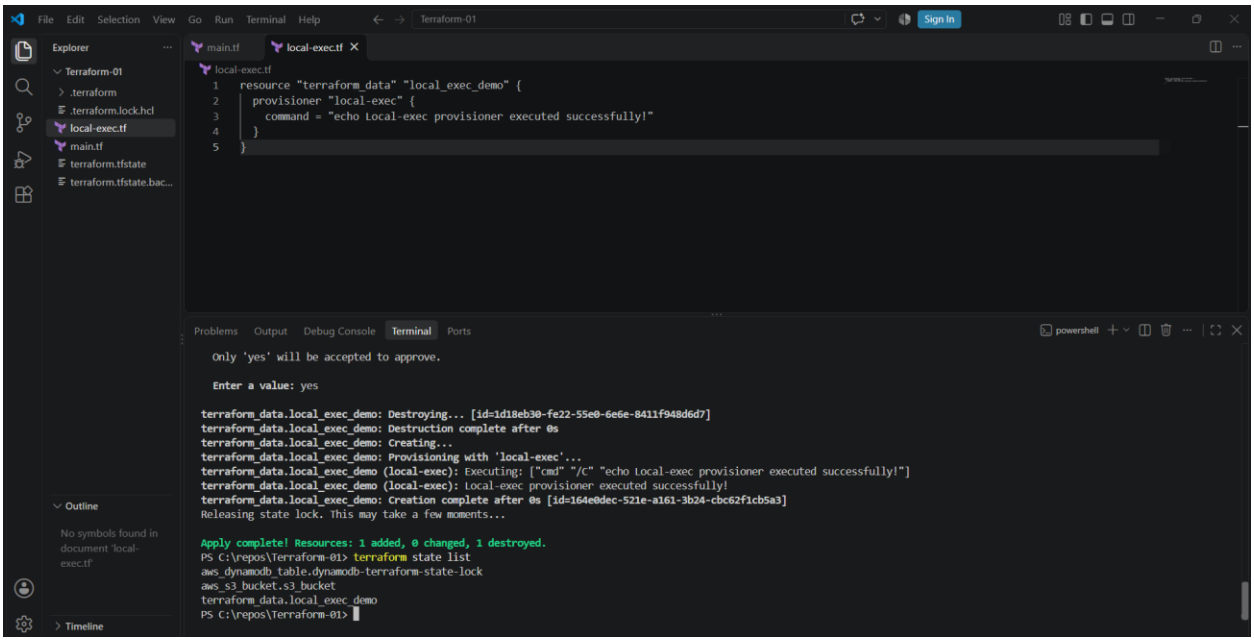
### Step 5: Verified the resource state

```
terraform state list
```

The output confirmed that `terraform_data.local_exec_demo` remained tracked, along with the existing S3 bucket and DynamoDB table.

The replacement resource received a new ID:

```
164e0dec-521e-a161-3b24-cbc62f1cb5a3
```




# 4. Observation

- Terraform identified the selected resource for replacement.
- The plan showed one resource to add and one to destroy.
- The existing resource was destroyed successfully.
- Terraform created the replacement with a new resource ID.
- The `local-exec` provisioner executed again during resource creation.
- The S3 bucket and DynamoDB table remained unchanged.
- No additional AWS infrastructure was created by this exercise.

# 5. Conclusion

Successfully performed Terraform resource replacement using the modern `-replace` option.

Learned that `terraform plan -replace` previews the replacement, while `terraform apply -replace` performs the replacement after confirmation. This approach is the recommended alternative to the deprecated `terraform taint` command.

Resource replacement can be useful when a resource needs to be recreated to recover from a problem or refresh its state. However, replacing real production resources may cause downtime or data loss, so the execution plan and resource impact must be reviewed before applying.

# Task: Terraform Debugging Using Debug Logs

## 1. Objective

To enable Terraform debug logging, generate a log file, and inspect detailed diagnostic messages to understand how Terraform validates configuration and initializes provider plugins.

## 2. Commands Used

### Enable debug logging

```
$env:TF_LOG = "DEBUG"
$env:TF_LOG_PATH = "terraform-debug.log"
```

- `TF_LOG = "DEBUG"` enables detailed Terraform diagnostic logging.
- `TF_LOG_PATH` specifies the file where Terraform writes the logs.

### Validate the configuration

```
terraform validate
```

Output:

```
Success! The configuration is valid.
```

### Verify the log file

```
Get-Item terraform-debug.log
```

The log file was created successfully in:

```
C:\repos\Terraform-01
```

Its reported size was 7,861 bytes.

### Inspect the log file

```
Get-Content terraform-debug.log -Tail 20
```

This displayed the last 20 lines of the debug log.

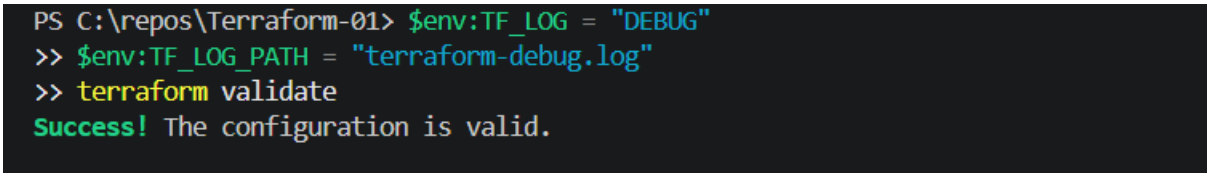
### Disable debug logging

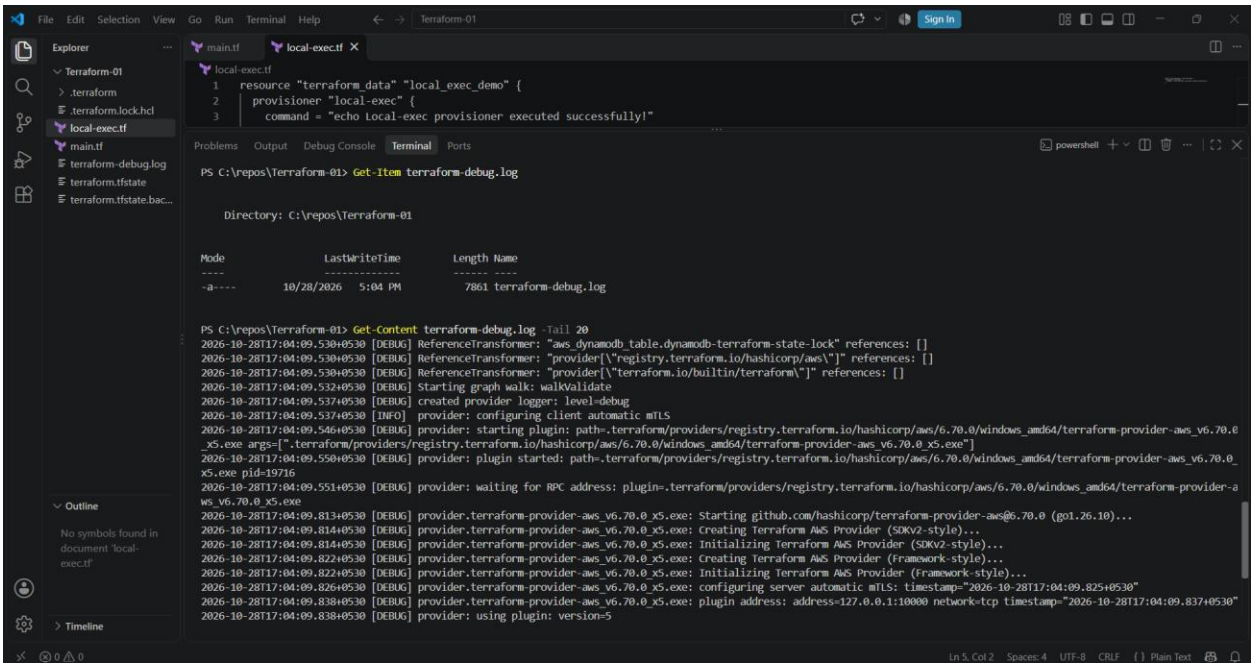
```
Remove-Item Env:TF_LOG
Remove-Item Env:TF_LOG_PATH
```

These commands remove the debug environment variables from the current PowerShell session. The generated log file remains available for inspection.

## 3. Steps Followed

1. Opened PowerShell in the existing Terraform project directory.
2. Enabled debug logging by setting `TF_LOG` to `DEBUG`.
3. Configured `TF_LOG_PATH` to save the output in `terraform-debug.log`.
4. Executed `terraform validate` to check the Terraform configuration.
5. Verified that the debug log file was created using `Get-Item`.
6. Inspected the last 20 lines using `Get-Content`.
7. Reviewed the provider initialization and process-exit messages.
8. Prepared to disable debug logging after completing the investigation.






# 4. Observation

- Terraform validation completed successfully.
- The debug log file was generated with detailed diagnostic information.
- The log showed Terraform starting and initializing the AWS provider plugin.
- The log included provider communication and process-exit messages.
- No validation error was reported.
- The provider process exited after completing its work; the displayed exit messages did not indicate a failure in this execution.

# 5. Conclusion

Successfully enabled Terraform debug logging, generated a log file, and inspected the diagnostic output.

Learned that debug logs help investigate Terraform issues by showing internal operations, provider initialization, and execution details. These logs are useful when troubleshooting configuration or provider problems that cannot be understood from the normal terminal output alone.

# Task 3: Create one EC2 instance with httpd installed using a Terraform script.

## 1. Objective

To create one AWS EC2 instance using Terraform and automatically install Apache HTTP Server using EC2 user data. Configure a security group to allow HTTP traffic on port 80 and access the website through a browser.

## 2. Tools and Technologies Used

- Cloud provider: AWS
- Infrastructure as Code: Terraform
- Operating system: Ubuntu Linux
- Web server: Apache HTTP Server (`apache2`)
- Instance type: `t3.micro`
- AWS region: `us-east-1`
- Configuration file: `main.tf`
- Terraform state: Local state

## 3. Steps Followed

### Step 1: Open the working directory

Opened PowerShell and worked in:

```
cd C:\repos\Terraform-01
```

### Step 2: Configure the AWS provider

Configured the AWS provider with region `us-east-1` and provider version constraint `~> 6.0`.

### Step 3: Create a security group

Created `asif-terraform-apache-sg` with these rules:

- Inbound: Allow TCP port 80 from `0.0.0.0/0`.
- Outbound: Allow all outbound traffic.

This allows users to access the web server over HTTP.

## Step 4: Define the EC2 instance

Configured one EC2 instance with:

- AMI: `ami-0d6dd9e8265d48906`
- Instance type: `t3.micro`
- Security group: `aws_security_group.apache_sg.id`

## Step 5: Install Apache using user data

Used the `user_data` argument to run a startup script that:

1. Updates the package lists.
2. Installs Apache.
3. Enables and starts the Apache service.
4. Creates a custom HTML welcome page in `/var/www/html/index.html`.

## Step 6: Initialize Terraform

Command:

```
terraform init -reconfigure
```

Because the previous S3 backend configuration was removed for this task, the old `.terraform` initialization cache also had to be cleared. Terraform was then initialized with local state.

## Step 7: Validate the configuration

Command:

```
terraform validate
```

Result: `Success! The configuration is valid.`

## Step 8: Review the execution plan

Command:

```
terraform plan
```

Terraform planned to create two resources: one EC2 instance and one security group.

### Step 9: Create the infrastructure

Command:

```
terraform apply
```

Reviewed the plan and entered `yes` when prompted.

Result:

```
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

### Step 10: Access the website

Opened the public IP address in a browser:

http://3.92.51.203

Apache uses HTTP port 80. The website is expected to display the custom welcome page after the startup script completes.

## 4. Complete Terraform Code (`main.tf`)

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_security_group" "apache_sg" {
```

```
  name        = "asif-terraform-apache-sg"
  description = "Allow HTTP traffic to Apache"

  ingress {
   description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name        = "asif-terraform-apache-sg"
    Environment = "Terraform-Lab"
  }
}

resource "aws_instance" "apache_server" {
  ami                         = "ami-0d6dd9e8265d48906"
  instance_type               = "t3.micro"
  vpc_security_group_ids      = [aws_security_group.apache_sg.id]
  user_data_replace_on_change = true

  user_data = <<-EOF
    #!/bin/bash
    set -eux

    apt-get update
    apt-get install -y apache2
    systemctl enable --now apache2

    cat > /var/www/html/index.html <<'HTML'
```

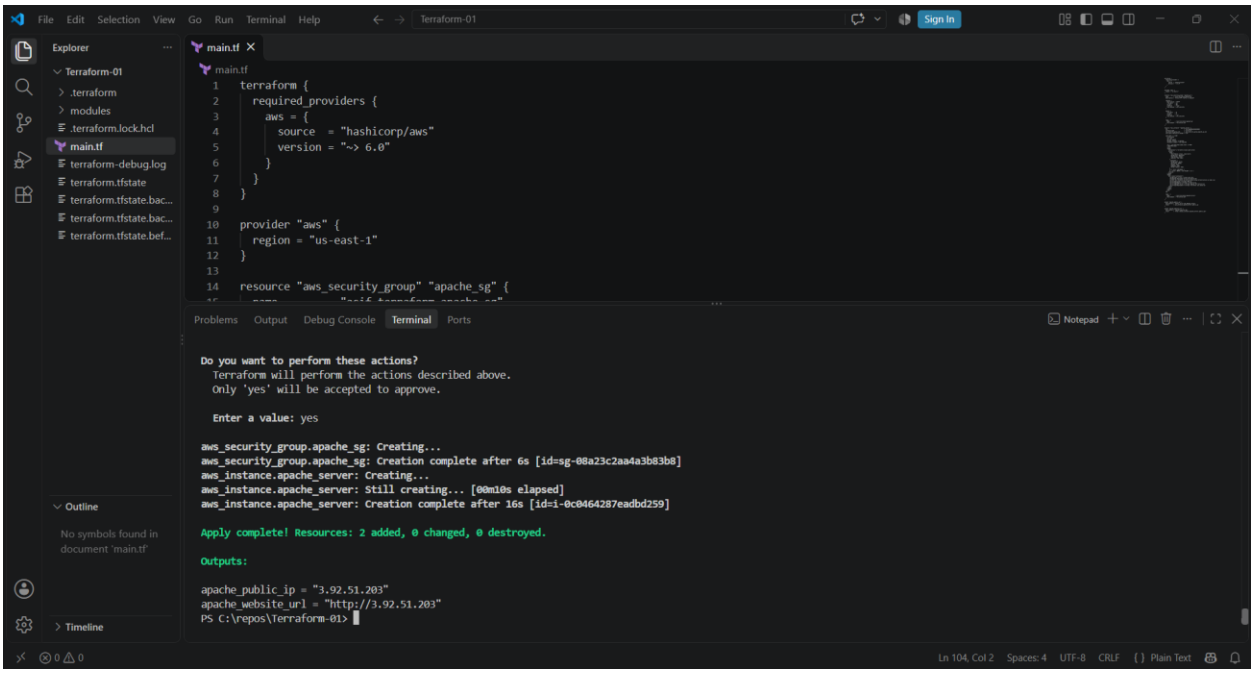
```
    <!DOCTYPE html>
    <html>
    <head>
      <title>Asif's Terraform Cloud Lab</title>
      <style>
        body {
          font-family: Arial, sans-serif;
          background: #eef3f8;
          text-align: center;
          padding: 70px 20px;
        }
        .container {
          background: white;
          max-width: 650px;
          margin: auto;
          padding: 35px;
          border-radius: 12px;
        }
        h1 { color: #1769aa; }
        p { color: #444; line-height: 1.8; }
      </style>
    </head>
    <body>
      <div class="container">
        <h1>Welcome to Asif's Cloud Lab!</h1>
        <h2>Apache Deployment Using Terraform</h2>
        <p>This EC2 instance was provisioned using Infrastructure as
Code.</p>
        <p><strong>Cloud:</strong> AWS</p>
        <p><strong>Region:</strong> us-east-1</p>
        <p><strong>Web Server:</strong> Apache HTTP Server</p>
        <p><strong>Deployment:</strong> Terraform user_data</p>
      </div>
    </body>
    </html>
    HTML
  EOF

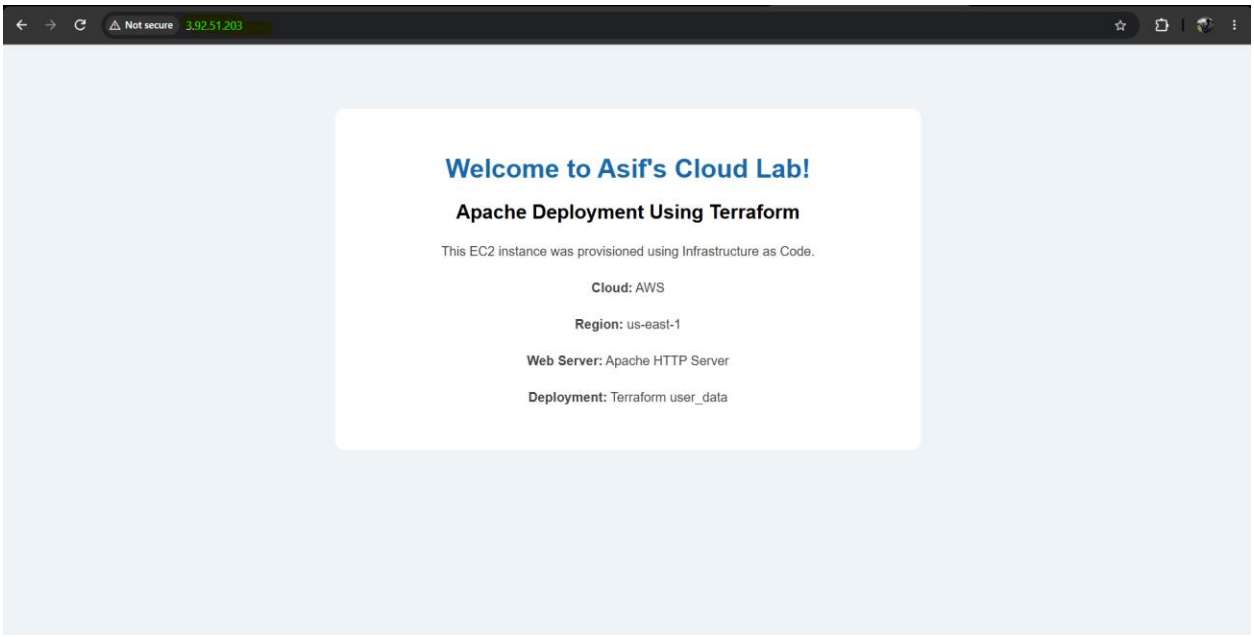
  tags = {
```

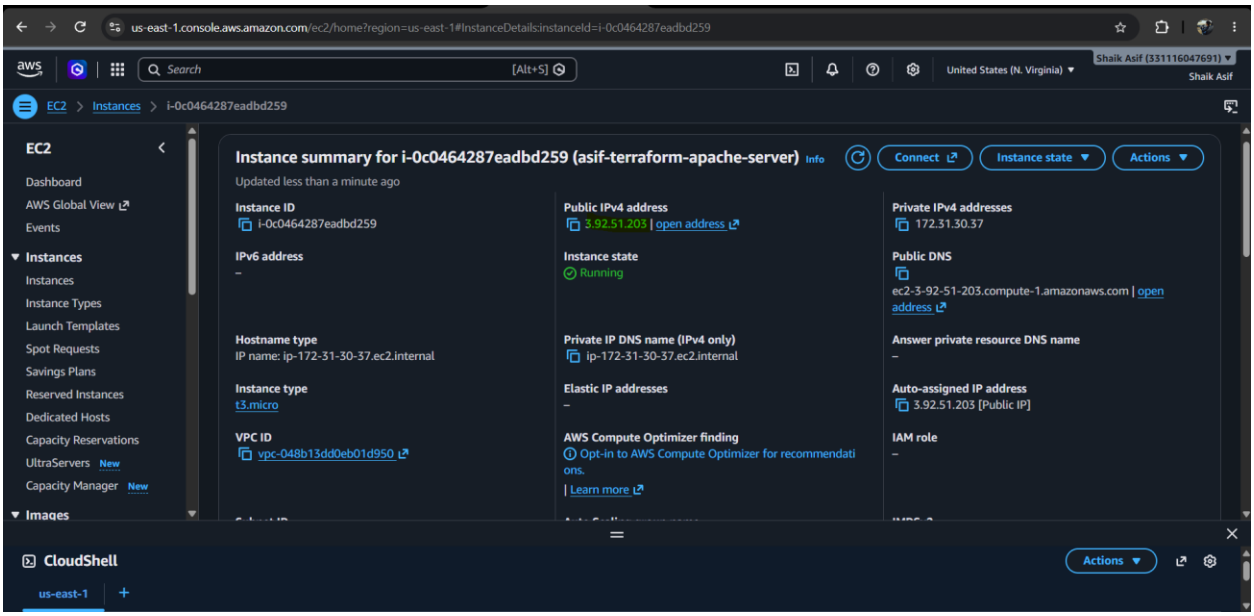
```
    Name        = "asif-terraform-apache-server"
    Environment = "Terraform-Lab"
  }
}

output "apache_public_ip" {
  description = "Public IP of the Apache server"
  value       = aws_instance.apache_server.public_ip
}

output "apache_website_url" {
  description = "Apache website URL"
  value       = "http://${aws_instance.apache_server.public_ip}"
}
```






# 5. Observation

Terraform successfully created the infrastructure.

| Property | Result |
|---|---|
| EC2 instance ID | `i-0c0464287eadbd259` |
| Public IP | `3.92.51.203` |
| Instance type | `t3.micro` |

| | |
|---|---|
| Region | `us-east-1` |
| Security group ID | `sg-08a23c2aa4a3b83b8` |
| HTTP port | `80` |
| Resources created | 2 |

The EC2 instance was provisioned, and the Apache installation commands were configured to execute automatically during instance startup.

## 6. Conclusion

Successfully provisioned an EC2 instance and its security group using Terraform. Used EC2 user data to automate Apache installation and create a custom web page. This task demonstrates Infrastructure as Code, AWS security group configuration, automated software installation, and Terraform's `init`, `validate`, `plan`, and `apply` workflow.

# Task 4: Set up S3 as backend for task 3.

## 1. Objective

To configure Amazon S3 as a remote backend for Terraform and migrate the existing local state file to S3. This allows Terraform state to be stored centrally and accessed from different environments.

## 2. Steps Used

**Step 1: Created an S3 bucket**

Created the S3 bucket using AWS CLI in the `us-east-1` region.

```
aws s3api create-bucket --bucket s3backend-asif-terraform-2026 --
region us-east-1
```

**Step 2: Verified the bucket**

Checked the bucket contents using:

```
aws s3 ls s3backend-asif-terraform-2026 --region us-east-1
```

The command returned no output because the bucket was empty.

**Step 3: Created the backend configuration**

Created a file named `backend.tf` in the Terraform project directory and configured the S3 backend.

**Step 4: Migrated the existing Terraform state**

Executed:

```
terraform init -migrate-state
```

When Terraform asked whether to copy the existing local state to S3, entered `yes`.

Terraform successfully configured the S3 backend and initialized the required provider.

**Step 5: Verified the remote state file**

Executed:

```
aws s3 ls s3://s3backend-asif-terraform-2026/remote-state/ --region
us-east-1
```

The command confirmed that `terraform.tfstate` was present in the S3 bucket.

# 3. Code Used

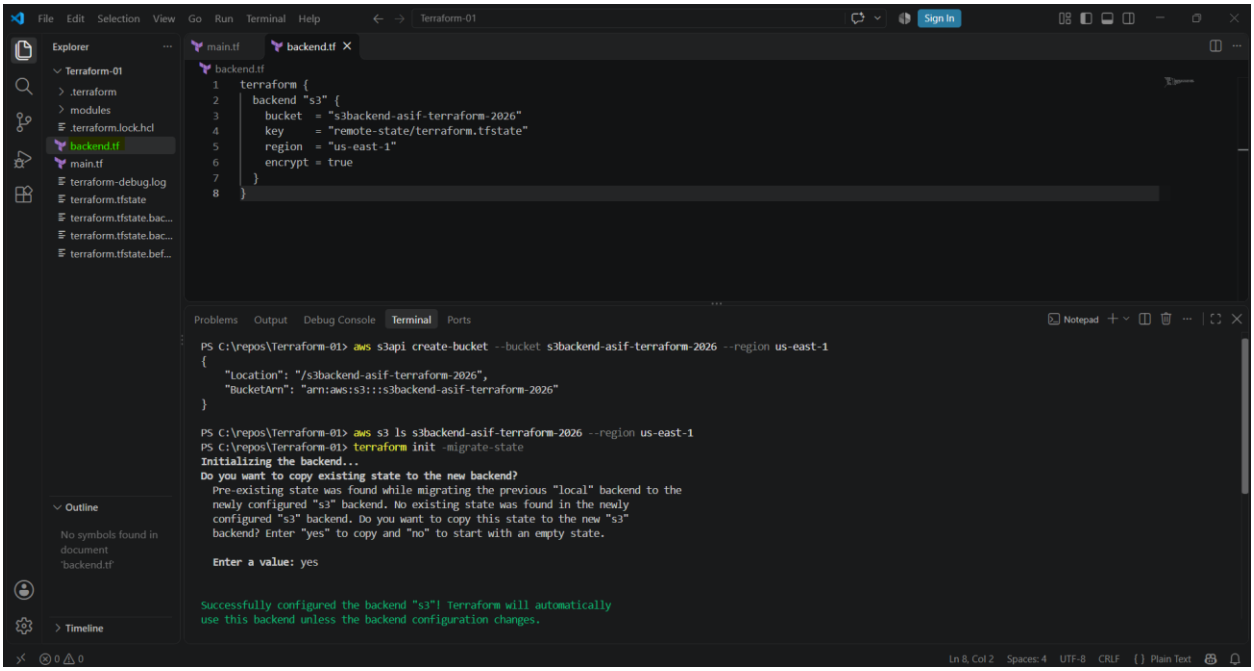
## backend.tf

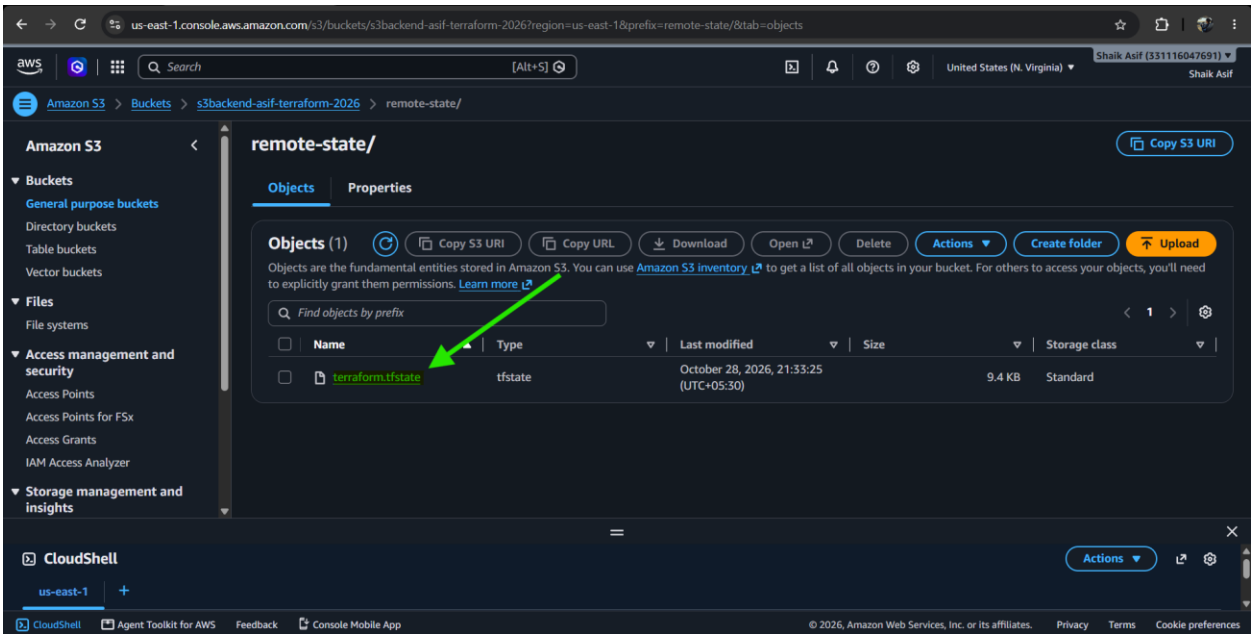
```
terraform {
  backend "s3" {
    bucket  = "s3backend-asif-terraform-2026"
    key     = "remote-state/terraform.tfstate"
    region  = "us-east-1"
    encrypt = true
  }
}
```

## Code Explanation

- **backend "s3":** Configures Amazon S3 as the Terraform state backend.

- **bucket:** Specifies the S3 bucket where the state file is stored.
- **key:** Specifies the path and name of the state file inside the bucket.
- **region:** Specifies the AWS region where the bucket is located.
- **encrypt:** Enables server-side encryption for the backend's state object.



# 4. Observation

- The S3 bucket was created successfully.
- Terraform initialized successfully with the S3 backend.
- The existing local state was migrated to S3.
- The remote state file was verified at `remote-state/terraform.tfstate`.
- The existing EC2 instance and security group continued to be tracked by Terraform rather than being recreated.
- DynamoDB state locking was not configured in this task; it is part of Task 5.

# 5. Conclusion

Successfully configured Amazon S3 as the remote backend for Terraform and migrated the existing local state file to the S3 bucket. Terraform can now store its infrastructure state remotely, making it easier to share and manage state across environments.

## Task 5: Set up DynamoDB locking for task 3.

# 1. Objective

To configure Amazon DynamoDB for Terraform state locking while using Amazon S3 as the remote backend.

State locking prevents multiple users or Terraform processes from modifying the same state file simultaneously. This helps avoid conflicting changes and state corruption.

# 2. Steps Used

### Step 1: Created a DynamoDB table

Created the DynamoDB table using AWS CLI.

```
aws dynamodb create-table `
  --table-name terraform-state-lock-dynamo `
  --attribute-definitions AttributeName=LockID,AttributeType=S `
  --key-schema AttributeName=LockID,KeyType=HASH `
```

```
  --billing-mode PAY_PER_REQUEST `
  --region us-east-1
```

**Observation:** The table creation started successfully.

## Step 2: Verified the table status

Executed:

```
aws dynamodb describe-table --table-name terraform-state-lock-dynamo --region us-east-1 --query "Table.TableStatus" --output text
```

**Observation:** The table status was `ACTIVE`.

## Step 3: Updated the Terraform backend configuration

Opened `backend.tf` and added the DynamoDB table configuration to the existing S3 backend.

## Step 4: Reinitialized Terraform

Executed:

```
terraform init -reconfigure
```

**Observation:** Terraform successfully configured the S3 backend. A deprecation warning appeared for the `dynamodb_table` parameter, but initialization completed successfully.

## Step 5: Verified the Terraform state

Executed:

```
terraform state list
```

**Observation:** Terraform listed both existing resources:

- `aws_instance.apache_server`
- `aws_security_group.apache_sg`

This confirmed that the existing infrastructure remained tracked by Terraform.

### Step 6: Tested state locking

Executed:

```
terraform plan
```

**Observation:** Terraform successfully acquired the state lock, refreshed the existing resources, reported no infrastructure changes, and released the state lock.

# 3. Code Used

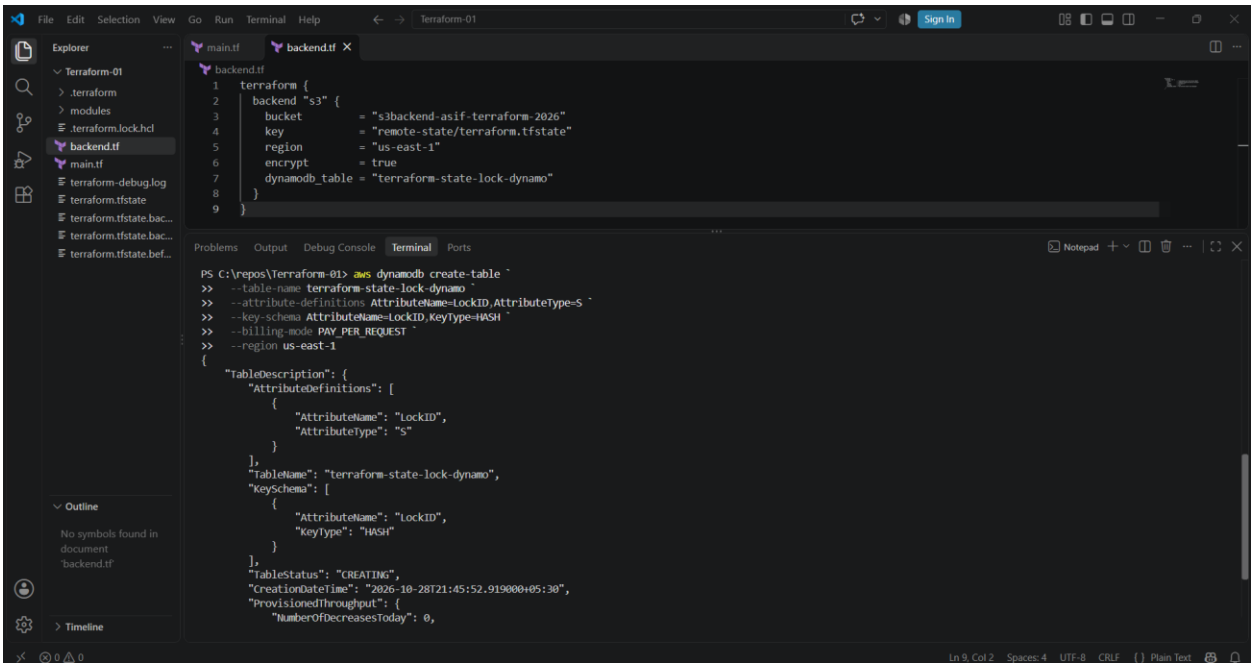
## `backend.tf`

```
terraform {
  backend "s3" {
    bucket         = "s3backend-asif-terraform-2026"
    key            = "remote-state/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-state-lock-dynamo"
  }
}
```

### Code Explanation

- **backend "s3":** Configures Amazon S3 as the remote Terraform state backend.
- **bucket:** Specifies the S3 bucket storing the state file.
- **key:** Specifies the state file path inside the bucket.
- **region:** Specifies the AWS region.
- **encrypt:** Enables server-side encryption for the S3 state object.
- **dynamodb_table:** Specifies the DynamoDB table Terraform uses for state locking.

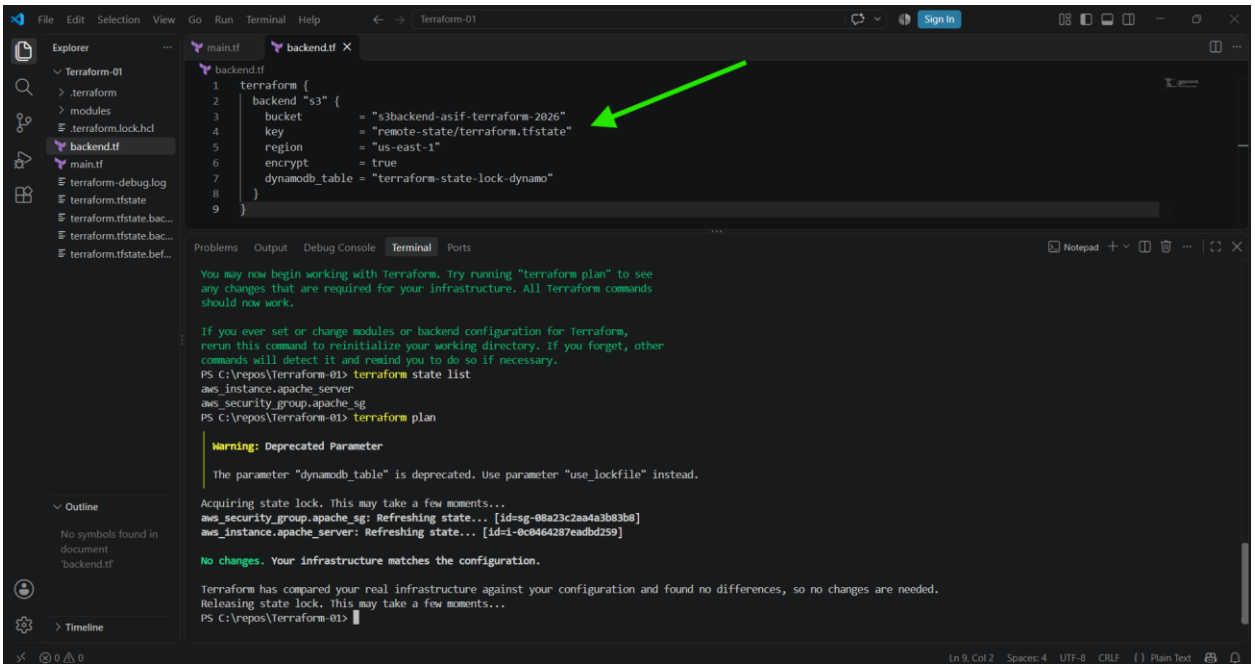
**Note:** Terraform reports that `dynamodb_table` is deprecated in favor of S3-native locking using `use_lockfile`. The configuration above is retained because this assignment specifically requires DynamoDB locking.



backend.tf

```
terraform {
  backend "s3" {
    bucket         = "s3backend-asif-terraform-2026"
    key            = "remote-state/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-state-lock-dynamo"
  }
}
```

```
PS C:\repos\Terraform-01> aws dynamodb create-table `
>>   --table-name terraform-state-lock-dynamo `
>>   --attribute-definitions AttributeName=LockID,AttributeType=S `
>>   --key-schema AttributeName=LockID,KeyType=HASH `
>>   --billing-mode PAY_PER_REQUEST `
>>   --region us-east-1
{
    "TableDescription": {
        "AttributeDefinitions": [
            {
                "AttributeName": "LockID",
                "AttributeType": "S"
            }
        ],
        "TableName": "terraform-state-lock-dynamo",
        "KeySchema": [
            {
                "AttributeName": "LockID",
                "KeyType": "HASH"
            }
        ],
        "TableStatus": "CREATING",
        "CreationDateTime": "2026-10-28T21:45:52.919000+05:30",
        "ProvisionedThroughput": {
            "NumberOfDecreasesToday": 0,
```



backend.tf

```
terraform {
  backend "s3" {
    bucket         = "s3backend-asif-terraform-2026"
    key            = "remote-state/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-state-lock-dynamo"
  }
}
```

```
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\repos\Terraform-01> terraform state list
aws_instance.apache_server
aws_security_group.apache_sg
PS C:\repos\Terraform-01> terraform plan

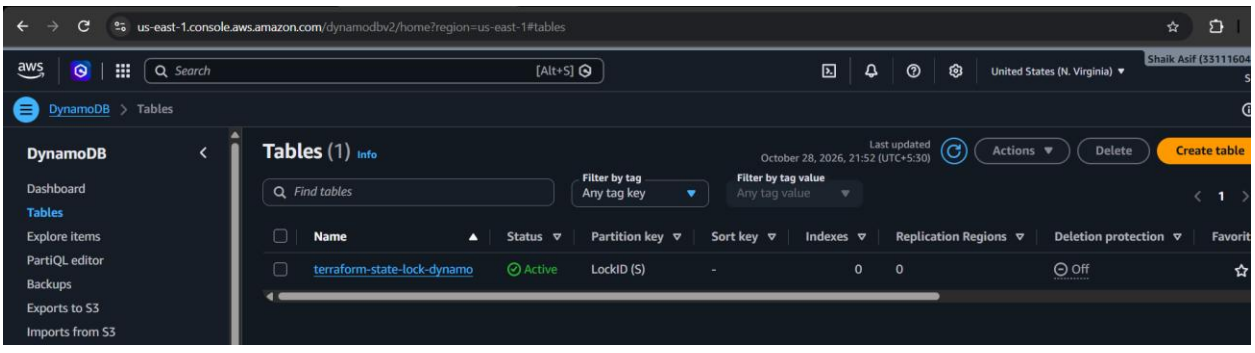
  Warning: Deprecated Parameter

  The parameter "dynamodb_table" is deprecated. Use parameter "use_lockfile" instead.

Acquiring state lock. This may take a few moments...
aws_security_group.apache_sg: Refreshing state... [id=sg-08a23c2aa4a3b83b8]
aws_instance.apache_server: Refreshing state... [id=i-0c0464287eadbd259]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
Releasing state lock. This may take a few moments...
PS C:\repos\Terraform-01>
```



# 4. Observation

1. The DynamoDB table `terraform-state-lock-dynamo` was created successfully.
2. The table status changed to `ACTIVE`.
3. Terraform successfully reinitialized with the updated backend configuration.
4. The EC2 instance and security group remained present in the Terraform state.
5. `terraform plan` acquired and released the state lock successfully.
6. Terraform reported: `No changes. Your infrastructure matches the configuration.`
7. No infrastructure resources were created or destroyed during verification.

# 5. Conclusion

Successfully configured DynamoDB state locking with an S3 remote backend for Terraform. Verified that Terraform could acquire and release the state lock while checking the existing infrastructure.

This setup helps prevent concurrent Terraform operations from modifying the same state simultaneously and supports safer infrastructure management.